Refatoração com Padrões de Projeto GoF

em Sistema Legado Solidity - Allo Protocol V2

QXD0068 - Reuso de Software

Universidade Federal do Ceará - Campus Quixadá

Aluno: Francisco Josias da Silva Batista
Matricula: 542167
Professor: Prof. Francisco Victor da Silva Pinheiro



Sumário da Apresentação

- Sistema e problemas do legado
- Os 4 padrões GoF aplicados
- Padrão 1: Facade
- Padrão 2: Strategy
- Padrão 3: Factory Method
- Padrão 4: State
- Benefícios arquiteturais
- Resultados e conclusão

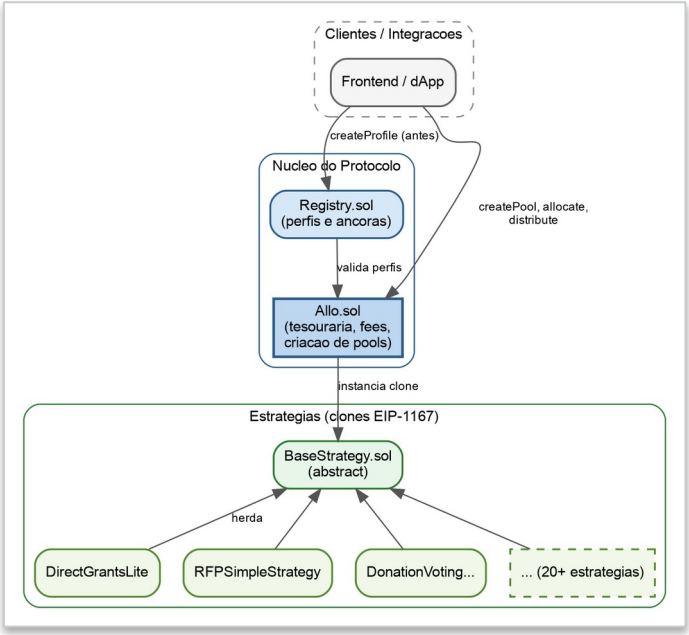
1. Sistema e problemas do legado

- Allo Protocol V2 - alocação de recursos para DAOs na Ethereum

Problemas identificados

- **God Class:** Allo.sol com 830+ linhas e múltiplas responsabilidades
- **Acoplamento alto:** negócio misturado com infraestrutura.
- **Taxas hardcoded:** Fee fixo em percentFee
- **Duplicação:** validações de estado repetidas em vários métodos.
- **Interface dispersa:** Operações comuns exigem várias transações.
- Para criar uma campanha são 2 transações (Registry + Allo)
- Clonagem EIP-1167 embarcada no contrato principal

Arquitetura



2. Os 4 padrões GoF aplicados

- Cada padrão ataca um problema arquitetural específico

Padrão	Categoria	Problema Resolvido	Contrato Principal
Facade	Estrutural	Interface dispersa para criar pools	AlloStarterFacade.sol
Strategy	Comportamental	Taxas hardcoded no núcleo	FlatFeeStrategy.sol + IFeeStrategy
Factory Method	Criacional	Clonagem acoplada ao Allo.sol	StrategyFactory.sol + IStrategyFactory
State	Comportamental	Validações duplicadas de estado	LifecycleState.sol + estados concretos

Princípios aplicados: SOLID, baixo acoplamento, alta coesão e inversão de dependência

3. Padrão 1: Facade - Estrutural

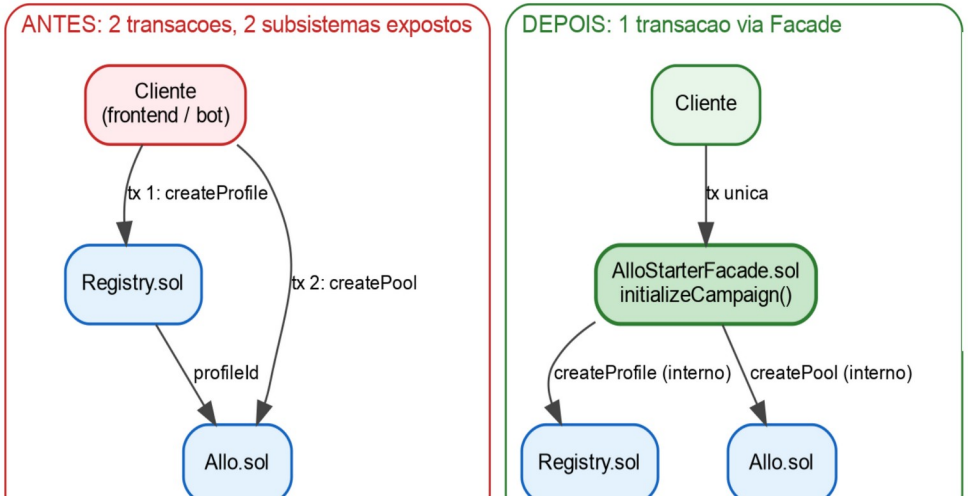
- Criação de campanha: de 2 transações para 1

Problema

- Cliente orquestra createProfile + createPool
- Precisa conhecer 2 contratos diferentes
- 2x gas e mais chance de erro de integração

Solução

- AlloStarterFacade encapsula os 2 subsistemas
- initializeCampaign() faz tudo em uma tx
- Cliente só passa os parâmetros essenciais



4. Padrão 2: Strategy - Comportamental

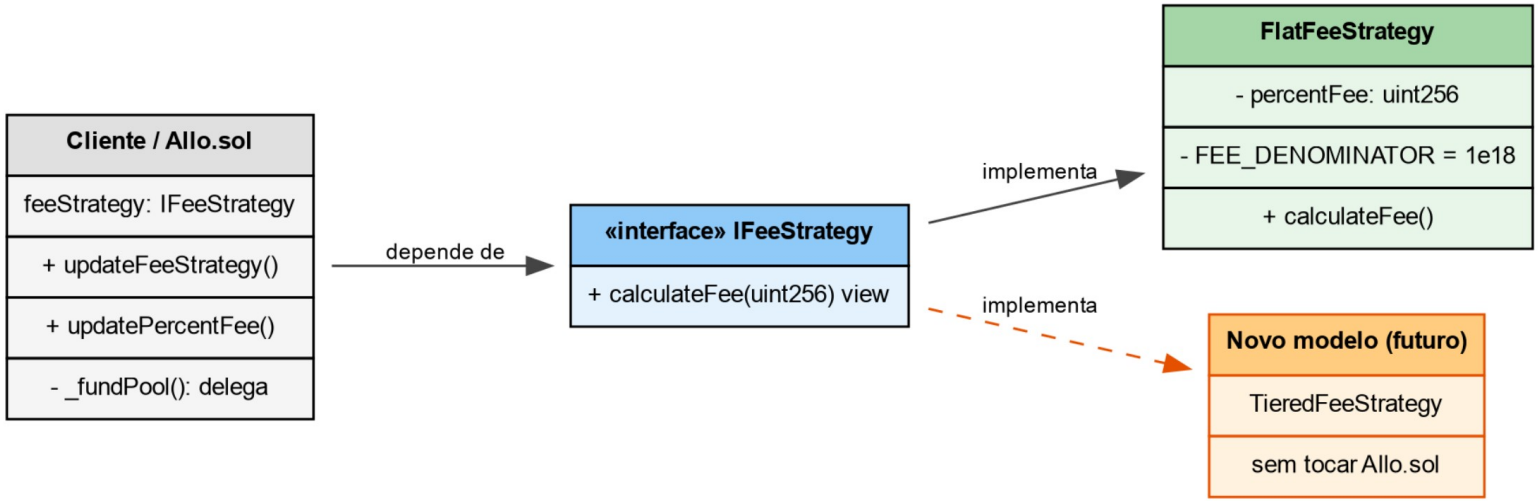
- Cálculo de taxas plugável, sem redeploy do Allo.sol

Problema

- percentFee fixo em 20% dentro de _fundPool
- Mudar o modelo econômico exigiria editar o núcleo
- Viola o princípio Aberto/Fechado (OCP)

Solução

- Interface IFeeStrategy com calculateFee()
- FlatFeeStrategy reproduz o cálculo legado
- Troca via updateFeeStrategy() (onlyOwner)



5. Padrão 3: Factory Method - Criacional

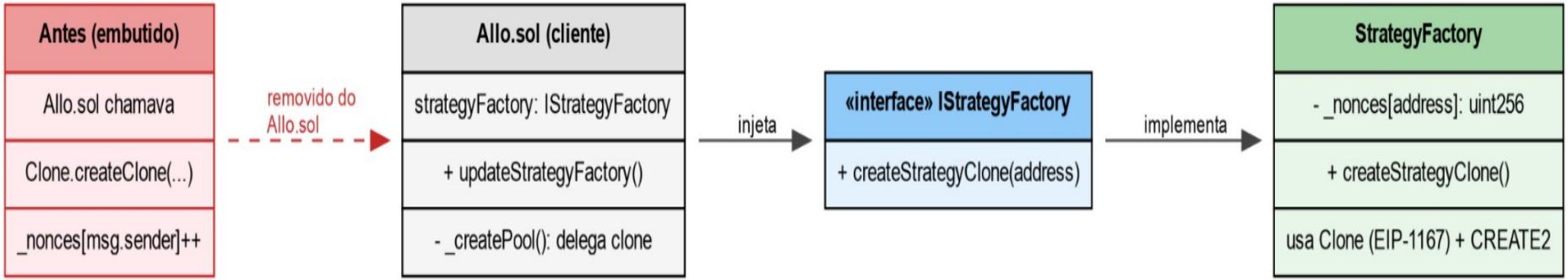
- Clonagem de estratégias extraída do contrato principal

Problema

- Clone.createClone + CREATE2 dentro de Allo.sol
- Bytecode do núcleo cresce a cada responsabilidade
- Viola a Responsabilidade Única (SRP)

Solução

- IStrategyFactory (produto abstrato)
- StrategyFactory guarda o nonce e clona
- Allo.sol delega em uma linha via injeção



6. Padrão 4: State - Comportamental

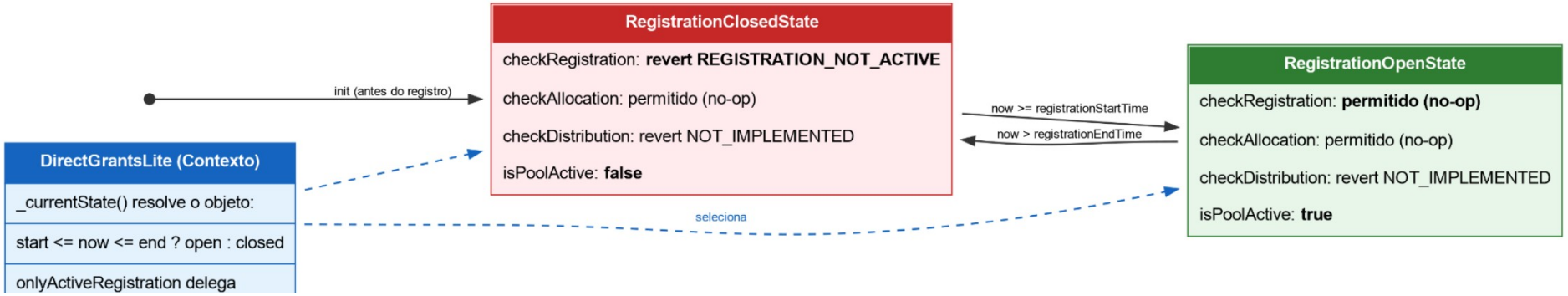
- Ciclo de vida da pool modelado com objetos de estado

Problema

- Janela de registro checada 2x (revert e boolean)
- Cadeias de if temporais repetidas
- Adicionar fase nova = copiar as mesmas condições

Solução

- LifecycleState abstrato + enum PoolState
- RegistrationOpenState / RegistrationClosedState
- _currentState() resolve lazy, sem storage extra



7. Benefícios arquiteturais

- O que a refatoração destravou para reuso e manutenção

Reutilização

- Factory reutilizável em outros protocolos
- Estratégias de taxas plugáveis
- Estados independentes do negócio

Extensibilidade

- TieredFee sem tocar no Allo.sol
- Novos estados sem refatorar métodos
- Novas fábricas: proxy, beacon.

Manutenibilidade

- Menos duplicação de código
- Complexidade ciclomática menor
- Responsabilidades separadas (SRP)

SOLID

- SRP: uma responsabilidade por contrato, OCP: aberto para extensão, fechado para modificação, LSP: estados e estratégias intercambiáveis, ISP: interfaces granulares e específicas, DIP: dependência de interfaces, não de implementações.

8. Resultados

- Comparação antes/depois e validação com a suíte de testes

Métrica	Antes	Depois	Melhoria
Linhas em Allo.sol	702	797	+13% (custo de Strategy/Factory)
Chamadas para criar campanha	2	1	50% menos transações
Modelos de taxas	1 (hardcoded)	N (plugáveis via IFeeStrategy)	Extensível sem modificar Allo.sol
Lógica de lifecycle duplicada	~9 linhas (2 checagens)	0 (delegada ao State)	100%
Testes automatizados	1115	1147	+32 novos testes

Validação técnica

- Suíte completa: 1147 testes (Foundry)
- 32 testes unitários novos para os padrões.
- Sem quebra de compatibilidade
- 1 commit por padrão - refatoração incremental

Conclusão

- Padrões GoF funcionam em Solidity desde que se respeitem gas e imutabilidade

O que mudou

- Facade: criação de campanha em 1 transação
- Strategy: modelo de taxas plugável (OCP)
- Factory: clonagem isolada da fábrica (SRP)
- State: ciclo de vida sem duplicação
- Núcleo do Allo.sol menor e mais coeso

Ganhos para reuso

- Novas taxas sem redeploy do Allo.sol
- Estados reutilizáveis entre estratégias
- Factory pronta para proxy/beacon no futuro
- Suíte legada verde = rede de segurança
- Dificuldades: storage layout e custo de gas

Repositório: http://github.com/josiasdev/allo-v2

Muito Obrigado!

Universidade Federal do Ceará - Campus Quixadá

Francisco Josias da Silva Batista – 542167
Repositório: http://github.com/josiasdev/allo-v2

